DS 642: Applications of Parallel Computing

Shahriar Afkhami

Week 11:
Fast Fourier Transform (FFT) And Parallel FFT

Trigonometric Interpolation

Trigonometric Polynomial Approximation

- The family of functions $\{1/2, \cos(x), \ldots, \cos(nx), \sin(x), \ldots, \sin((n-1)x)\}$ is orthogonal on $[-\pi, \pi]$.
- For a continuous function, $f(x)$, on $[-\pi, \pi]$, we seek a continuous least squares approximation of the form

$$S_n(x) = a_0/2 + a_n \cos(nx) + \sum_{k=1}^{n-1} (a_k \cos(kx) + b_k \sin(kx))$$

  with respect to $w(x) = 1$.
- The limit of $S_n(x)$ when $n \to \infty$ is called the Fourier series of $f(x)$.

Discrete Trigonometric Approximation

- Suppose that a collection $2m$ paired data points $\{(x_i, y_i)\}_{i=0}^{2m-1}$ on equally spaced interval $[-\pi, \pi]$ is given. I.e. $x_i = -\pi + (i/m)\pi$, $i = 0, \ldots, 2m-1$.
- We must evaluate the coefficients of the interpolatory polynomial (discrete least squares polynomial)

$$S_m(x) = a_0/2 + a_m \cos(mx)/2 + \sum_{k=1}^{m-1} (a_k \cos(kx) + b_k \sin(kx))$$

- The interpolation of $2m$ data points by the direct-calculation technique requires approximately $(2m)^2$ multiplications and $(2m)^2$ additions.

Discrete Trigonometric Approximation

- Evaluating the constants a_k and b_k is directly related to the discrete Fourier transform procedure of computing the complex coefficients c_k in

$$c_k = \sum_{j=0}^{2m-1} (y_j e^{ik\pi j/m}), \quad S_m(x) = \frac{1}{m} \sum_{k=0}^{2m-1} c_k e^{ikx}$$

for $k = 0, \ldots, 2m-1$. Once c_k have been determined, the polynomial coefficients can be recovered readily;
$c_k = (-1)^k m(a_k + ib_k)$.

Computing the Coefficients

$$c_k = \sum_{j=0}^{2m-1} (y_j e^{ik\pi j/m}), \quad S_m(x) = \frac{1}{m} \sum_{k=0}^{2m-1} c_k e^{ikx}$$

But only half the terms in the sum need to be computed:

$$c_k + c_{m+k} = 2 \sum_{j=0}^{m-1} (y_{2j} e^{ik\pi j/(m/2)})$$

$$c_k - c_{m+k} = 2e^{ik\pi m} \sum_{j=0}^{m-1} (y_{2j+1} e^{ik\pi j/(m/2)})$$

The new sums have the same structure as the initial sum, so we can apply the same operations-reducing scheme again. Since $m = 2^n$, we can keep applying this $n + 1$ times, and we will have $O(m \log_2 m)$ operations.

Fast Fourier Transform: Discrete Fourier Transform

- The transformation

$$\mathbf{X}_k = \sum_{n=0}^{N-1} (\mathbf{x}_n e^{-2\pi ikn/N})$$

for $k = 0, \ldots, N-1$, can be written as

$$\mathbf{X} = \mathbf{Fx}$$

- $\mathbb{F}$ is an $N \times N$ matrix, called Fourier matrix, defined as

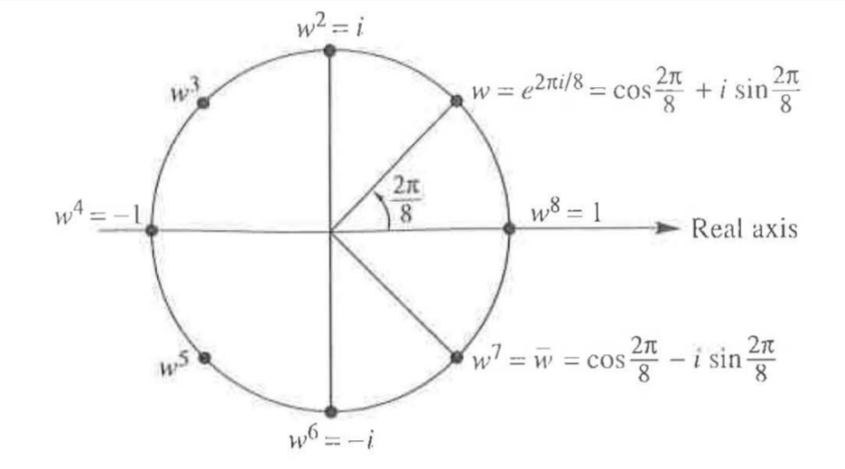
$$F[i,j] = \omega^{(jk)}$$

where $\omega = e^{(2\pi i/N)} = \cos(2\pi/N) + i\sin(2\pi/N)$, and $0 \leq j, k \leq N-1$.

we need N^2 multiplications and $N(N-1)$ additions.

(2D DFT of $N \times N$ matrix **V** is **FVF** – Do 1D DFT on all the columns independently, then all the rows)

Fast Fourier Transform



The Fourier matrix:

$$F_{N\times N} = \begin{bmatrix} 1 & \omega^0 & \omega^0 & \cdots & 1 \\ 1 & \omega^1 & \omega^2 & \cdots & \omega^{N-1} \\ 1 & \omega^2 & \omega^4 & \cdots & \omega^{2(N-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{N-1} & \omega^{2(N-1)} & \cdots & \omega^{(N-1)^2} \end{bmatrix}$$

which is a full matrix and $\omega^N = e^{(2\pi i)} = \cos(2\pi) + i\sin(2\pi) = 1$.

Fast Fourier Transform

inverse Fourier matrix:

$$F_{N\times N}^{-1} = \frac{1}{N}\begin{bmatrix} 1 & 1 & 1 & \cdots & 1 \\ 1 & \omega^{-1} & \omega^{-2} & \cdots & \omega^{-(N-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{-(N-1)} & \omega^{-2(N-1)} & \cdots & \omega^{-(N-1)^2} \end{bmatrix} = (\omega^{-jk})_{j,k=0}^{N-1}$$

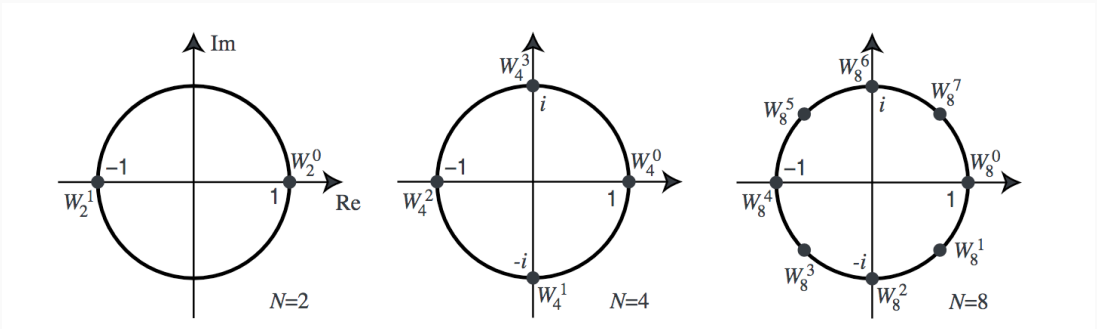
where $\omega^N = e^{(2\pi i)} = \cos(2\pi) + i\sin(2\pi) = 1$.

Fast Fourier Transform

For $N = 4$, for example:

$$F_{4\times 4} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & \omega^1 & \omega^2 & \omega^3 \\ 1 & \omega^2 & \omega^4 = 1 & \omega^6 \\ 1 & \omega^3 & \omega^6 & \omega^9 \end{bmatrix}$$

$$\omega^4 = e^{(2\pi i)} = \cos(2\pi) + i\sin(2\pi) = 1.$$
$$\omega^1 = e^{(2\pi i/4)} = \cos(2\pi/4) + i\sin(2\pi/4) = i.$$
$$\omega^2 = e^{(2\pi i/2)} = i^2 = -1.$$
$$\omega^3 = e^{(6\pi i/4)} = i^3 = -i.$$



Fast Fourier Transform

For $N = 4$, for example:

$$F_{4\times 4} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & \omega^1 = i & i^2 & i^3 \\ 1 & i^2 = -1 & i^4 = 1 & i^6 \\ 1 & i^3 = -i & i^6 & i^9 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & i & -1 & -i \\ 1 & -1 & 1 & -1 \\ 1 & -i & -1 & i \end{bmatrix}$$

Columns are orthogonal. (Columns of $F/2$ are orthonormal); $\bar{F}^T F = I$.

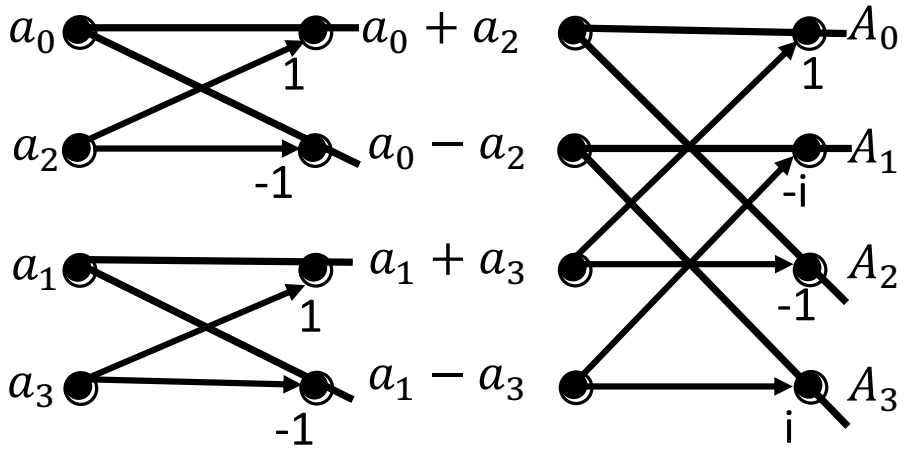
Fast Fourier Transform

$$A_0 = ($$

$$A_1 = ($$

$$A_2 = ($$

$$A_3 = ($$



we need 2 multiplications (by $\pm i$) and 8 additions.

Fast Fourier Transform

The key idea is that $\omega(N=4) = e^{(2\pi i/4)}$ is related to $(\omega(N=8))^2 = e^{2(2\pi i/8)} = \omega(N=4)$ For $N=4$, for example:

$$F_{8\times 8} = \begin{bmatrix} I & D \\ I & -D \end{bmatrix} \begin{bmatrix} F_{4\times 4} & 0 \\ 0 & F_{4\times 4} \end{bmatrix} P$$

$$D_{4\times 4} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \omega & 0 & 0 \\ 0 & 0 & \omega^2 & 0 \\ 0 & 0 & 0 & \omega^3 \end{bmatrix} P_{8\times 8} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

The formula above show that the DFT of N components could be converted to compute the DFT of two $N/2$ components, and some simple additions and multiplications.

MATLAB FFT

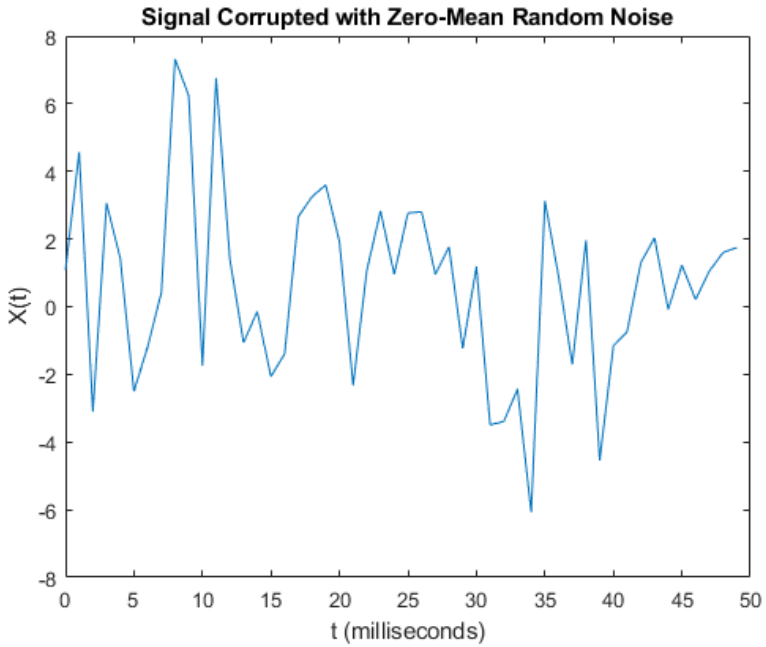
```
S = 0.7*sin(2*pi*50*t) + sin(2*pi*120*t);
```

Corrupt the signal with zero-mean white noise with a variance of 4.

```
X = S + 2*randn(size(t));
```

Plot the noisy signal in the time domain. It is difficult to identify the frequency components by loo

```
plot(1000*t(1:50),X(1:50))
title('Signal Corrupted with Zero-Mean Random Noise')
xlabel('t (milliseconds)')
ylabel('X(t)')
```



Compute the Fourier transform of the signal.

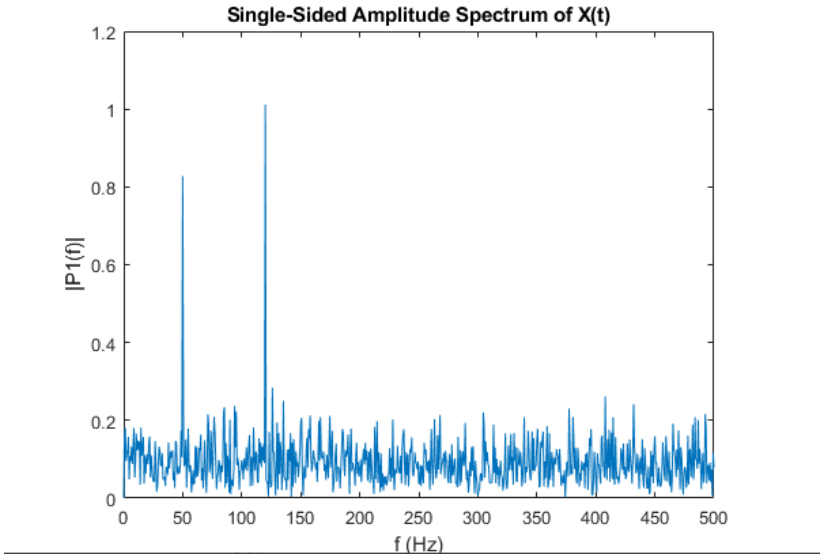
```
Y = fft(X);
```

Compute the two-sided spectrum P2. Then compute the single-sided spectrum P1 based on P2 and the

```
P2 = abs(Y/L);
P1 = P2(1:L/2+1);
P1(2:end-1) = 2*P1(2:end-1);
```

Define the frequency domain f and plot the single-sided amplitude spectrum P1. The amplitudes are nc

```
f = Fs*(0:(L/2))/L;
plot(f,P1)
title('Single-Sided Amplitude Spectrum of X(t)')
xlabel('f (Hz)')
ylabel('|P1(f)|')
```



The Poisson problem

Consider the Poisson equation

$$-\nabla^2 u = f \quad \text{on} \quad \Omega$$

In electrostatics, for example, the differential forms for the electric field $\boldsymbol{E}$ are

$$\nabla \cdot \boldsymbol{E} = 4\pi\rho,$$

$$\nabla \times \boldsymbol{E} = 0,$$

where ρ is the charge density. It follows that the electric field $\boldsymbol{E}$ can be expressed as the gradient of a scalar field ϕ, i.e., $\boldsymbol{E} = -\nabla\phi$. Hence,

$$\nabla \cdot \boldsymbol{E} = -\nabla \cdot \nabla\phi = -\nabla^2\phi = 4\pi\rho.$$

We need to specify boundary conditions for u on the domain boundary.

The one-dimensional Poisson problem

We consider here the Poisson equation (or diffusion equation) in one space dimension,

$$-u_{xx} = f \qquad \text{in } \Omega = (0,1)$$

and with homogeneous boundary conditions,

$$u(0) = u(1) = 0.$$

Let u_i be an approximation to $u(x_i)$, $i = 1, \ldots, n-1$, and let $f_i = f(x_i)$. A finite difference approximation of the Poisson problem can then be expressed as

$$-\left(\frac{u_{i+1} - 2u_i + u_{i-1}}{h^2}\right) = f_i, \qquad i = 1, \ldots, n-1,$$

$$u_0 = 0,$$

$$u_n = 0.$$

The one-dimensional Poisson problem

In matrix form, this system can be expressed as

$$\underbrace{\begin{pmatrix} 2 & -1 & & & \\ -1 & 2 & -1 & & \\ & \ddots & \ddots & \ddots & \\ & & -1 & 2 & -1 \\ & & & -1 & 2 \end{pmatrix}}_{\boldsymbol{A}} \underbrace{\begin{pmatrix} u_1 \\ u_2 \\ \vdots \\ u_{n-2} \\ u_{n-1} \end{pmatrix}}_{\boldsymbol{u}} = h^2 \underbrace{\begin{pmatrix} f_1 \\ f_2 \\ \vdots \\ f_{n-2} \\ f_{n-1} \end{pmatrix}}_{\boldsymbol{f}},$$

or as $\boldsymbol{Au} = h^2 \boldsymbol{f}$.

A 2D version can be derived analogously.

The one-dimensional Poisson problem

We want to solve $\boldsymbol{A}\boldsymbol{u} = \boldsymbol{b}$:

- Consider eigenvalue/eigenvector decomposition of $\boldsymbol{A} = F\,D\,F^T$.
- F is very similar to FFT (imaginary part):
  - $F(i,j) = (2/(n+1))^{1/2} \sin(jk\pi/(n+1))$
- D is diagonal matrix of eigenvalues:
  - $D(i,j) = 2(1 - \cos(j\pi/(n+1)))$
- Substitute $\boldsymbol{A} = F\,D\,F^T$ in $\boldsymbol{A}\boldsymbol{u} = \boldsymbol{b}$
  - $F\,D\,F^T\boldsymbol{u} = \boldsymbol{b}$
- Perform
  - $F^T\,F\,D\,F^T\boldsymbol{u}\,F = F^T\,\boldsymbol{b}\,F$
- Then solve: $D\,\boldsymbol{x}' = \boldsymbol{b}'$, where $\boldsymbol{x}' = F^T\boldsymbol{u}\,F$ and $\boldsymbol{b}' = F^T\,\boldsymbol{b}\,F$
  - where $\boldsymbol{b}'$ is the FFT on $\boldsymbol{b}$
  - and to get $\boldsymbol{u}$, perform inverse FFT on $\boldsymbol{x}'$.

Algorithms for 2D (3D) Poisson Equation

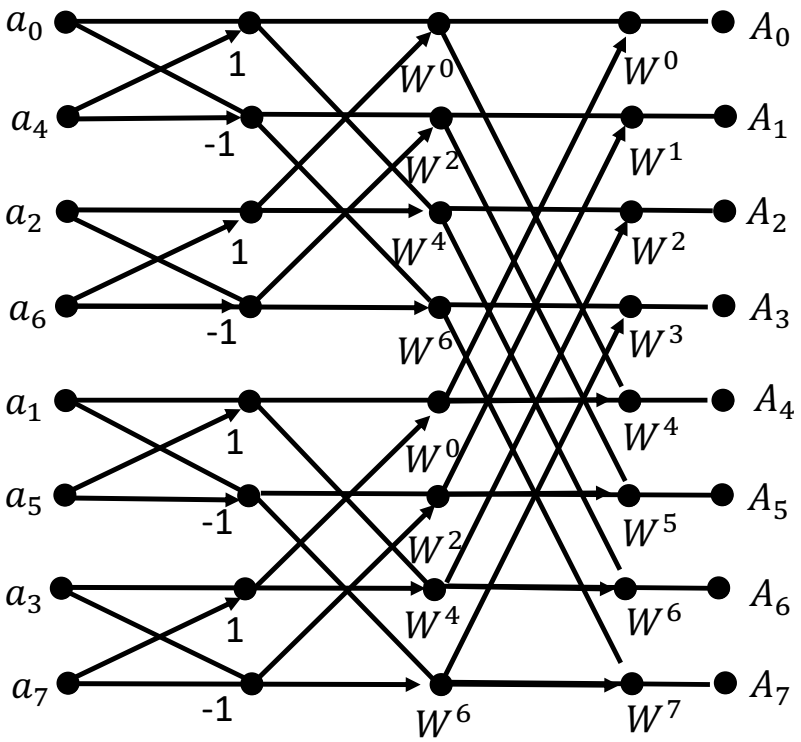
Algorithms for 2D (3D) Poisson Equation (N = n^2 (n^3) vars)

Algorithm	Serial	PRAM	Memory	#Procs
° Dense LU	N^3	N	N^2	N^2
° Band LU	N^2 ($N^{7/3}$)	N	$N^{3/2}$ ($N^{5/3}$)	N ($N^{4/3}$)
° Jacobi	N^2 ($N^{5/3}$)	N ($N^{2/3}$)	N	N
° Explicit Inv.	N^2	log N	N^2	N^2
° Conj.Gradients	$N^{3/2}$ ($N^{4/3}$)	$N^{1/2(1/3)}$ *log N	N	N
° Red/Black SOR	$N^{3/2}$ ($N^{4/3}$)	$N^{1/2}$ ($N^{1/3}$)	N	N
° Sparse LU	$N^{3/2}$ (N^2)	$N^{1/2}$	N*log N ($N^{4/3}$)	N
➡ ° **FFT**	**N*log N**	**log N**	**N**	**N**
° Multigrid	N	$\log^2$ N	N	N
° **Lower bound**	**N**	**log N**	**N**	

The Fast Fourier Transform (FFT) Algorithm

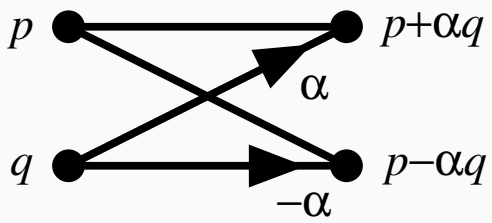
Below is a diagram of an 8-point

The FFT is a fas
computing the D
2-point DFT and
generalize them
16-point, ..., 2^r-p
FFT algorithm.
To compute the
sequence would
multiplies and a
algorithm comp
$O(N \log N)$ mult



Butterflies and Bit-Reversal

The FFT algorithm decomposes the DFT into $\log_2 N$ stages, each of which consists of $N/2$ *butterfly* computations. Each butterfly takes two complex numbers p and q and computes from them two other numbers, $p + \alpha q$ and $p - \alpha q$, where α is a complex number. Below is a diagram of a butterfly operation:



In the diagram of the 8-point FFT above, note that the inputs aren't in normal order: $a_0, a_1, a_2, a_3, a_4, a_5, a_6, a_7$, they're in the bizarre order: $a_0, a_4, a_2, a_6, a_1, a_5, a_3, a_7$. Why this sequence?

Butterflies and Bit-Reversal

Below is a table of j and the index of the jth input sample, n_j:

j	0	1	2	3	4	5	6	7
n_j	0	4	2	6	1	5	3	7
j base 2	000	001	010	011	100	101	110	111
n_j base 2	000	100	010	110	001	101	011	111

The pattern is obvious if j and n_j are written in binary (last two rows of the table). Observe that each n_j is the *bit-reversal* of j. The sequence is also related to breadth-first traversal of a binary tree.

It turns out that this FFT algorithm is simplest if the input array is rearranged to be in bit-reversed order.

Butterflies and Bit-Reversal

The re-ordering can be done in one pass through the array *a*:

```
for j = 0 to N-1
    nj = bit_reverse(j)
    if (j < nj) swap a[j] and a[nj]
```

FFT Explained Using Matrix Factorization

The 8-point DFT can be written as a matrix product, where we let $W = W_8 = e^{-i\pi/4} = (1-i)/\sqrt{2}$:

$$\begin{pmatrix} A_0 \\ A_1 \\ A_2 \\ A_3 \\ A_4 \\ A_5 \\ A_6 \\ A_7 \end{pmatrix} = \begin{pmatrix} W^0 & W^0 & W^0 & W^0 & W^0 & W^0 & W^0 & W^0 \\ W^0 & W^1 & W^2 & W^3 & W^4 & W^5 & W^6 & W^7 \\ W^0 & W^2 & W^4 & W^6 & W^0 & W^2 & W^4 & W^6 \\ W^0 & W^3 & W^6 & W^1 & W^4 & W^7 & W^2 & W^5 \\ W^0 & W^4 & W^0 & W^4 & W^0 & W^4 & W^0 & W^4 \\ W^0 & W^5 & W^2 & W^7 & W^4 & W^1 & W^6 & W^3 \\ W^0 & W^6 & W^4 & W^2 & W^0 & W^6 & W^4 & W^2 \\ W^0 & W^7 & W^6 & W^5 & W^4 & W^3 & W^2 & W^1 \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \\ a_4 \\ a_5 \\ a_6 \\ a_7 \end{pmatrix}$$

FFT Explained Using Matrix Factorization

Rearranging so that the input array a is bit-reversed and factoring the 8×8 matrix:

$$= \begin{pmatrix} 1 & \cdot & \cdot & \cdot & W^0 & \cdot & \cdot & \cdot \\ \cdot & 1 & \cdot & \cdot & \cdot & W^1 & \cdot & \cdot \\ \cdot & \cdot & 1 & \cdot & \cdot & \cdot & W^2 & \cdot \\ \cdot & \cdot & \cdot & 1 & \cdot & \cdot & \cdot & W^3 \\ 1 & \cdot & \cdot & \cdot & W^4 & \cdot & \cdot & \cdot \\ \cdot & 1 & \cdot & \cdot & \cdot & W^5 & \cdot & \cdot \\ \cdot & \cdot & 1 & \cdot & \cdot & \cdot & W^6 & \cdot \\ \cdot & \cdot & \cdot & 1 & \cdot & \cdot & \cdot & W^7 \end{pmatrix} \begin{pmatrix} 1 & \cdot & W^0 & \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & 1 & \cdot & W^2 & \cdot & \cdot & \cdot & \cdot \\ 1 & \cdot & W^4 & \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & 1 & \cdot & W^6 & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & 1 & \cdot & W^0 & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & 1 & \cdot & W^2 \\ \cdot & \cdot & \cdot & \cdot & 1 & \cdot & W^4 & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & 1 & \cdot & W^6 \end{pmatrix} \begin{pmatrix} 1 & W^0 & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ 1 & W^4 & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & 1 & W^0 & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & 1 & W^4 & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & 1 & W^0 & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & 1 & W^4 & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & 1 & W^0 \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & 1 & W^4 \end{pmatrix} \begin{pmatrix} a_0 \\ a_4 \\ a_2 \\ a_6 \\ a_1 \\ a_5 \\ a_3 \\ a_7 \end{pmatrix}$$

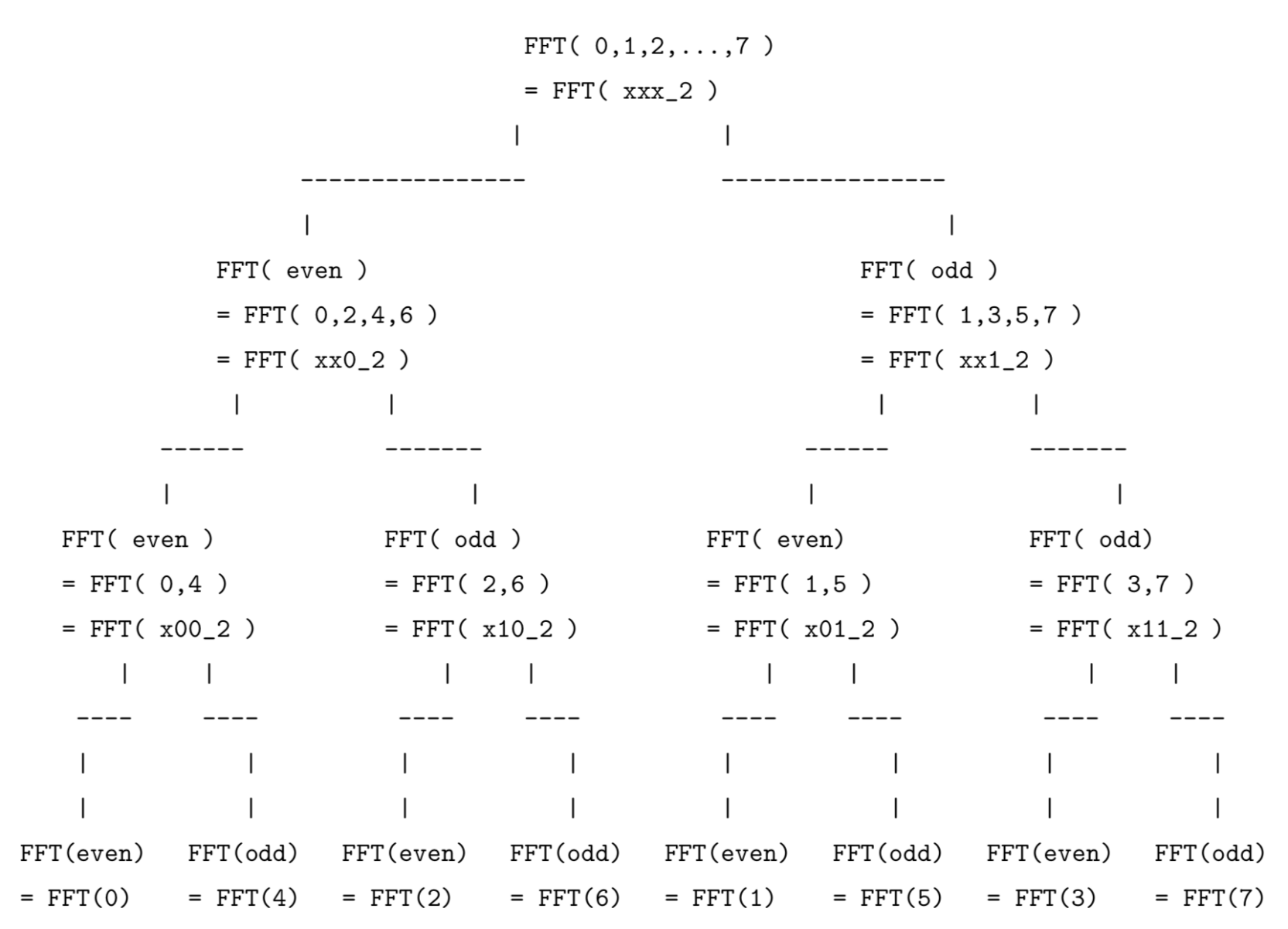
where "$\cdot$" means 0.

These are sparse matrices (lots of zeros), so multiplying by the dense (no zeros) matrix on top is more expensive than multiplying by the three sparse matrices on the bottom.

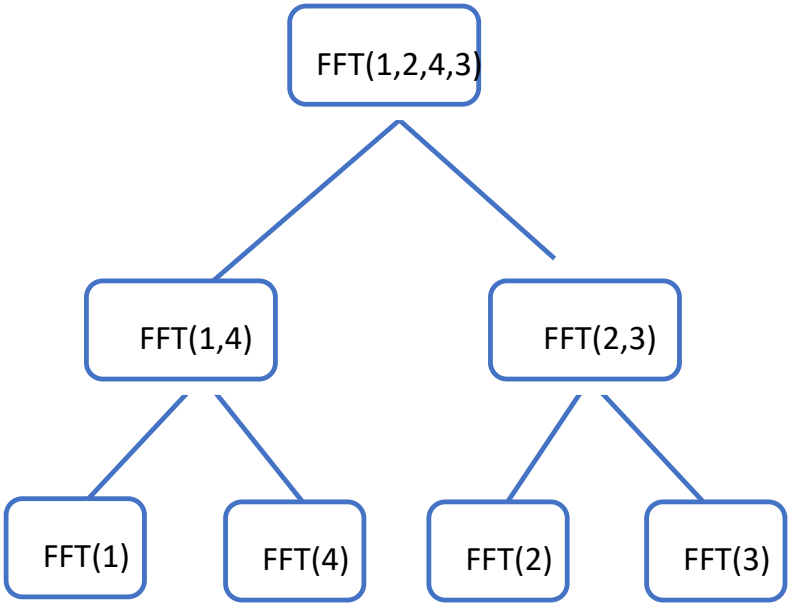
FFT Explained Using Matrix Factorization

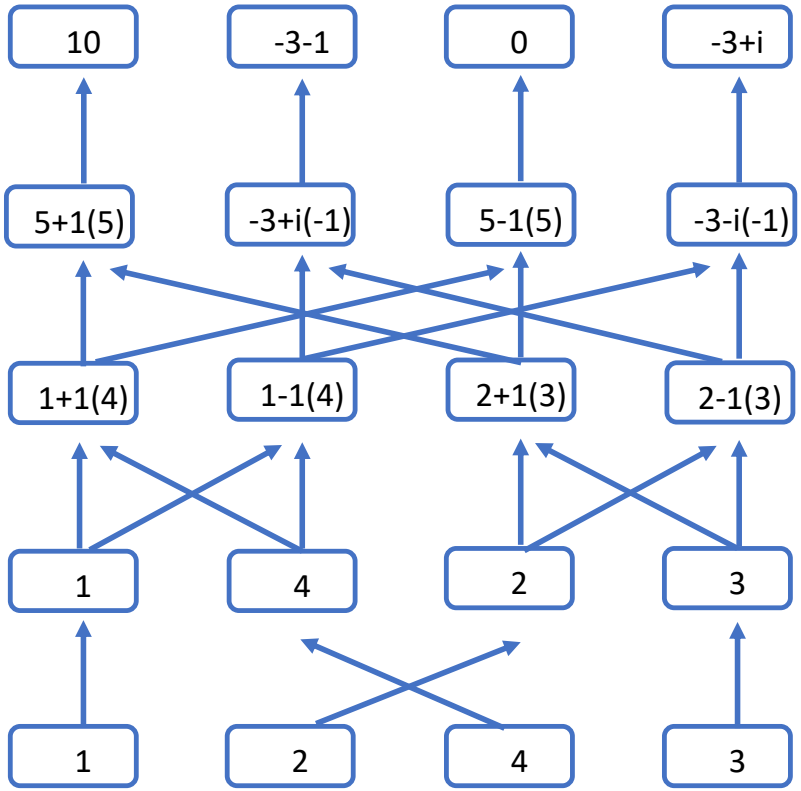
For $N = 2^r$, the factorization would involve r matrices of size $N \times N$, each with 2 non-zero entries in each row and column.

FFT Explained Using Matrix Factorization



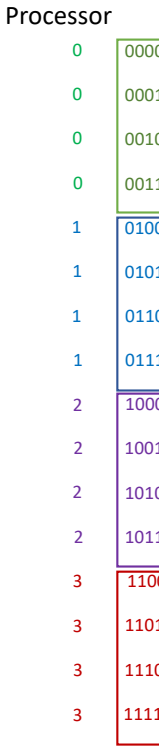
DFT Computation



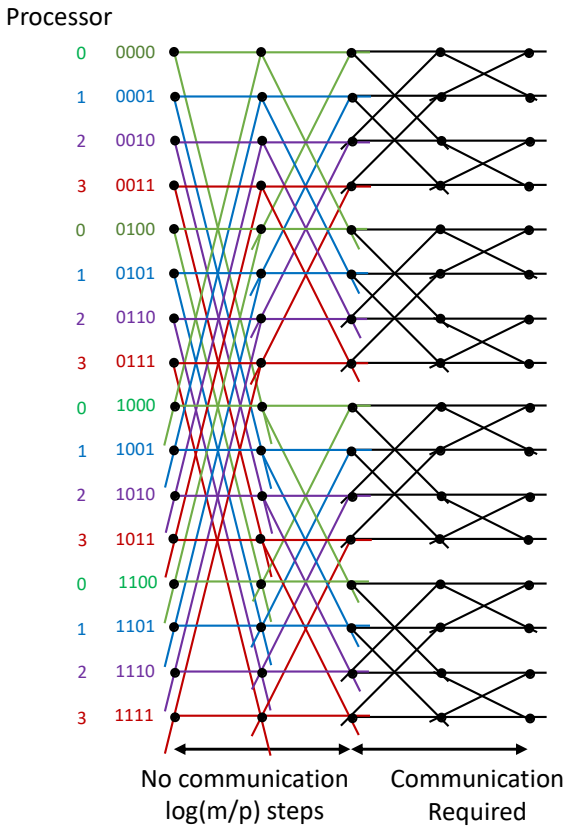


Parallel Fast Fourier Transform

left sequence is input and right sequence is output. Each



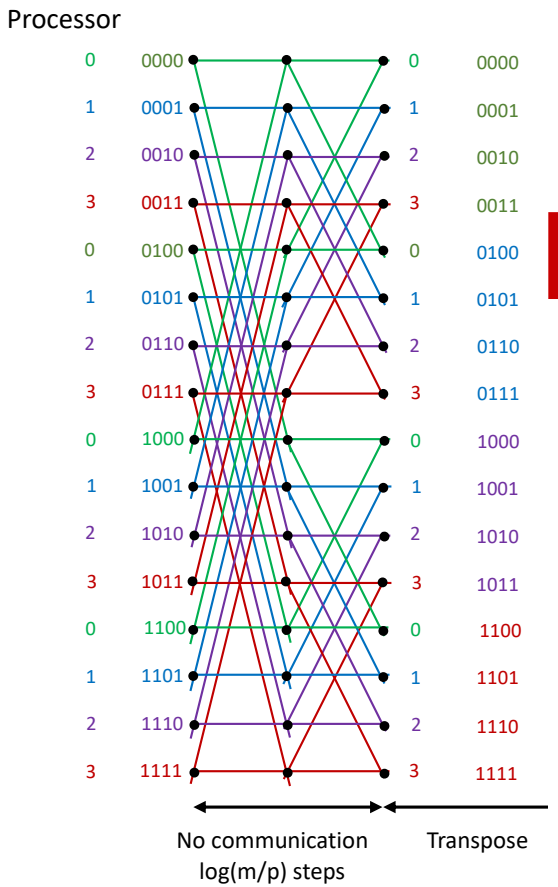
Block Layout

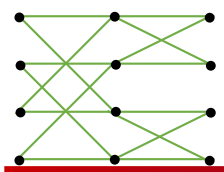


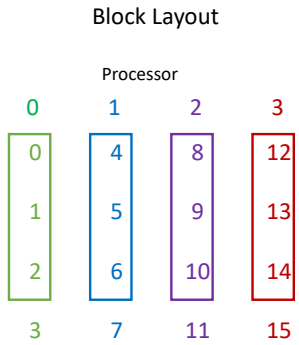
Cyclic Layout

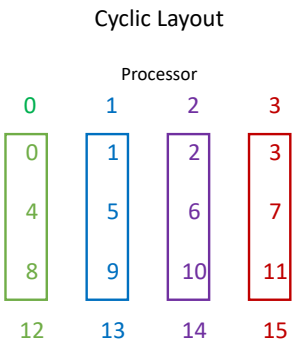
Parallel Fast Fourier Transform

If we start with a cyclic layout for first log(m/p) steps, there is no communication Then transpose the vector for last log(p) steps







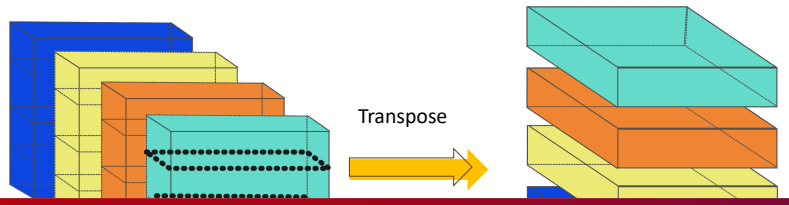


Higher Dimensional FFTs

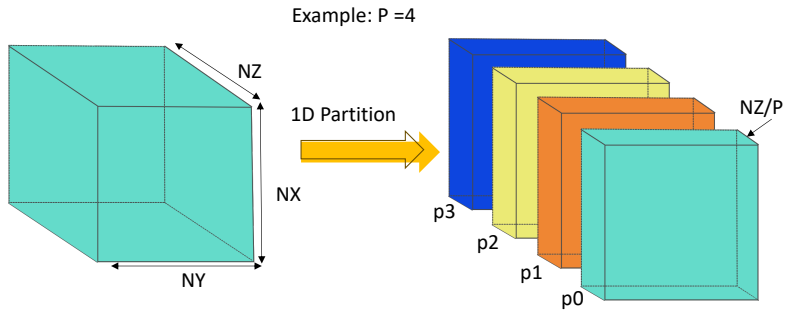
- FFT on 2 or more dimensions are defined as 1D FFTs on vectors in all dimensions.
- FFT on 2 or more dimensions are defined as 1D FFTs on vectors in all dimensions.
- 2D FFT does 1D FFTs on all rows and then all columns
  There are 3 obvious possibilities for the 2D FFT:
  - 2D blocked layout for matrix, using parallel 1D FFTs for each row and column
  - Block row layout for matrix, using serial 1D FFTs on rows, followed by a transpose, then more serial 1D FFTs
  - Block row layout for matrix, using serial 1D FFTs on rows, followed by parallel 1D FFTs on columns

  For a 3D FFT the options are similar 2 phases done with serial FFTs, followed by a transpose for 3rd. Can overlap communication with 2nd phase in practice.

Performing



NJIT Course: DS-642 NJIT Online



- Step 1: FFTs on the columns (all elements local)
- Step 2: FFTs on the rows (all elements local)
- FFTs in the Z-dimension (requires communication)
  - Perform a Global Transpose of the cube. Allows step 3 to continue

The Transpose

- Each processor has to scatter input domain to other processors
  - Every processor divides its portion of the domain into P pieces
  - Send each of the P-1 pieces to a different processor
- Three different ways to break up the messages
  - Packed Slabs (i.e. single packed “All-to-all” in MPI parlance) (3D)
  - Slabs (2D)
  - Pencils (1D)
- Going from approach Packed Slabs to Slabs to Pencils leads to
  - An order of magnitude increase in the number of messages
  - An order of magnitude decrease in the size of each message

Slabs and Pencils allow overlapping communication